

performance-testing-skills

Bộ Claude Skills tự động hoá kiểm thử hiệu năng với Grafana k6

Từ SLO đến Scorecard — có kiểm chứng độc lập ở mọi bước.

Lộ trình trình bày

- | | | |
|----|-------------------|---------------------------------------|
| 01 | Vấn đề | Vì sao cần một bộ skill có kiểm soát |
| 02 | Kiến trúc | Master – SubAgent Adversarial Pattern |
| 03 | Vòng đời | Sáu giai đoạn, sáu lệnh |
| 04 | Kiểm soát | Audit Gate & 4-Layer RCA |
| 05 | Thực chiến | Ví dụ EShop & tổng kết |
-

Khi chỉ quăng một câu cho AI

PROMPT SINH VIÊN HAY GỖ

“Viết script k6 test hiệu năng API checkout giúp tôi.”

01 · HARDCODE TOKEN

Dán cứng một token từ ví dụ

500 VU dùng chung một thẻ — test PASS, thực chất một user chạy 500 lần.

02 · PHÂN BỐ PHI THỰC TẾ

Chia đều 33 · 33 · 33%

Hành vi thật là 60 · 30 · 10 — báo cáo đẹp, không phải ngày cao điểm.

03 · RACE CONDITION

Không nghĩ ra kịch bản tranh mua

Hai người mua sản phẩm cuối — tồn kho âm chỉ lộ ở production.

04 · NGƯỠNG SLO DỄ DẪI

Chọn threshold rộng cho chắc

Không có baseline nào — test nào cũng pass vì vạch đích bị kéo gần.

AI không sai vì nó lười — nó sai vì không ai audit lại nó.

Bộ skill làm gì

Modular Claude Skills suite — 100% Grafana k6 — Master Agent thực thi, một Sub-Agent độc lập kiểm tra.

ĐẦU VÀO

Dự án của bạn

docs/ — HAR · spec · OpenAPI

CÁCH GẮN

→ **.claude/**

gìt submodule, không sửa mã dự án

BỘ SKILL

→ **performance-testing-skills**

6 skill · 6 slash-command

ĐẦU RA

→ **perf-test/**

script · log · report · scorecard

6

giai đoạn từ DEFINE đến SHIP

2

persona đối kháng nhau

3

cổng audit độc lập

Kiến trúc Master – Sub-Agent

MASTER · THỰC THI

perf-architect

Chạy toàn bộ DEFINE → SHIP.

Đàm phán SLO, sinh script k6, chạy test, viết report.

Có quyền làm mọi thứ — trừ việc tự công nhận kết quả của mình.

Không bao giờ tự chấm bài của chính mình.

SUB-AGENT · PHÁN QUYẾT

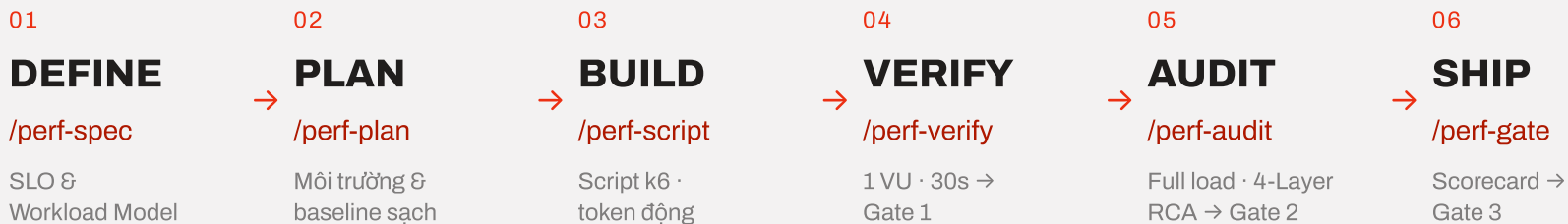
bottleneck-auditor

Không chạy k6, không viết script.

Chỉ soi bằng chứng thô: log, metric, SLO đã chốt.

Trả verdict **APPROVED / REJECTED** — Master không được ghi đè.

Vòng đời sáu giai đoạn



Ba giai đoạn cuối đều phải đi qua một cổng audit độc lập trước khi được đi tiếp.

Sáu skill, sáu lệnh

GIAI ĐOẠN	LỆNH	VIỆC CHÍNH
DEFINE	<code>/perf-spec</code>	Đọc HAR/OpenAPI → SLO + Workload Model
PLAN	<code>/perf-plan</code>	Kiểm tra môi trường, ép reset baseline sạch
BUILD	<code>/perf-script</code>	Sinh script k6, correlation token động
VERIFY	<code>/perf-verify</code>	Sanity 1 VU/30s, qua Audit Gate 1
AUDIT	<code>/perf-audit</code>	Full load + 4-Layer RCA, qua Audit Gate 2
SHIP	<code>/perf-gate</code>	Tổng hợp scorecard, qua Audit Gate 3

Vai trò của con người ở mỗi lệnh

LỆNH	NGƯỜI DÙNG CUNG CẤP / XÁC NHẬN	AGENT TỰ STOP KHI
/perf-spec	HAR · OpenAPI · spec đặt vào docs/	docs/ trống — không bịa spec từ trí nhớ
/perf-plan	Xác nhận trực tiếp bằng lời: target là test/staging, bên thứ ba ở chế độ sandbox	Chưa có xác nhận rõ ràng của người — URL “trông giống” staging không tính
GATE AN TOÀN		
/perf-script	Không cần thêm — Agent tự đọc PERF_SPEC.md	PERF_SPEC.md chưa tồn tại
/perf-verify	(tuỳ chọn) Prometheus/Grafana đang chạy hay không	Baseline của strategy chứa PASS
/perf-audit	Không cần thêm — chạy trên baseline đã có	VERIFY chứa APPROVED cho strategy này
/perf-gate	Không cần thêm — tổng hợp report đã duyệt	Còn strategy chứa APPROVED — không ký duyệt một phần

Bên trong một cổng audit

BƯỚC 01

Master soạn bản nháp

perf-architect tổng hợp kết quả đo được

BƯỚC 02

Đóng gói đúng → ba thứ

SLO mục tiêu · log thô · bản nháp — không kèm suy luận

- CONTEXT GỒ LẬP

BƯỚC 03

Sub-Agent đọc bằng chứng

Không thấy lý lẽ của Master nên không thể bị thuyết phục

BƯỚC 04

Verdict nhị phân

→ APPROVED hoặc REJECTED — không có vùng xám

✓ APPROVED

Lưu report → đi tiếp giai đoạn sau.

✗ REJECTED

Kèm REQUIRED_CORRECTIONS cụ thể → Master sửa & nộp lại → tối đa 3 lần, lần thứ tư escalate cho người dùng.

Gate 1 tại VERIFY · Gate 2 tại AUDIT (sâu nhất, có 4-Layer RCA) · Gate 3 tại SHIP (Final Sign-off).

Bốn lớp phân tích nguyên nhân

01	Transport	Độ trễ giả do load generator quá tải
02	Application	Event-loop lag, memory tăng không kiểm soát
03	Data	SQLITE_BUSY / SQLITE_LOCKED khi ghi đồng thời
04	Infrastructure	Chạm trần CPU/RAM do giới hạn container

Luôn kiểm tra đủ cả bốn lớp — CPU 100% chưa chắc là nguyên nhân gốc.

Ba lớp an toàn

GUARDRAIL 01

Target Authorization Gate

Con người xác nhận thủ công đây là test/staging. Không bao giờ nhả vào production hay domain bên thứ ba.

GUARDRAIL 02

Multi-Strategy Reset

Load → Stress → Spike → Soak chạy tuần tự. Mỗi strategy reset baseline riêng, evidence không ghi đè.

GUARDRAIL 03

Observability tùy chọn

Agent chỉ sinh file cấu hình Prometheus/Grafana. Việc docker compose up là của con người.

Sau ba lần REJECTED liên tiếp: dừng vòng lặp, escalate — không thử vô hạn.

Ví dụ thực chiến: EShop

Một luồng thật: search → cart → checkout.

/perf-spec

SLO p95 < 500ms · workload 60 · 30 · 10

/perf-plan

Staging được xác nhận · baseline reset sạch

/perf-script

Correlation token & cartID theo từng VU

/perf-verify

1 VU · 30s → Gate 1 APPROVED

/perf-audit

50 VU load → SQLITE_BUSY lộ diện ở lớp Data

/perf-gate

Scorecard cuối · Gate 3 sign-off

Ba điều cần nhớ

01

Không tự chấm bài

Mọi kết quả cần một Sub-Agent độc lập xác nhận.

02

Đo trước khi tối ưu

4-Layer RCA dựa trên bằng chứng chạy thật, không suy đoán.

03

An toàn trước tốc độ

Không có Target Authorization thì không có full-load test.

Hỏi & Đáp

Cảm ơn Thầy/Cô và các bạn đã theo dõi
phần trình bày của nhóm

Rất mong nhận được câu hỏi và góp ý — đặc biệt về cơ chế
Adversarial Audit và 4-Layer RCA.